

PRISM

(Programmable Reduction & Inference Search Manager)

Programmable Inference Control for PLN

Project Proposal — AGI Engineering Competition

Track A: Hybrid AGI — Cognitive, Neuro-Symbolic & Hyperon-Based AGI

iCog Labs

Proposed duration: 12 weeks (one quarter)

Author: **Abel Tesfa**

1. Executive Summary

PLN (Probabilistic Logic Networks) is Hyperon's core engine for turning stored facts into new, derived knowledge. Its logic is well specified; what is not solved is search — at almost any point in a derivation, PLN can typically apply many different valid rules to many different eligible premises, and there is no strong method for deciding which of those combinatorially many next steps is actually worth taking. This is not a problem we are inferring from the outside: Nil Geisweiller, the lead developer of PLN in Hyperon, has stated directly that a reasoning system geared toward AGI needs flexible, any-direction inference chaining, and that the requirement of programmable inference control is expected to be realized once minimal-MeTTa is completed — enabling a human or machine programmer to overwrite the reduction step and interject a conditional to prune or prioritize non-deterministic reduction.

PRISM is a learned search-guidance layer for PLN that does exactly this: it observes the set of legal next inference steps PLN itself generates, and ranks them by estimated usefulness, so PLN spends its search effort on the steps most likely to matter. It never touches soundness — every step it ranks highly still has to pass through PLN's own rule application and truth-value computation unchanged. PRISM only ever affects the order of search, never its validity.

This document explains what PLN's inference control problem actually is, why it remains open even inside Hyperon's own team, what PRISM is and how it works from first principles, and how it will be built, integrated, and measured.

2. Introduction & Motivation

Hyperon's AtomSpace can store facts, but a store of facts alone is not reasoning — an AGI-relevant system needs to derive new knowledge from what it already has, and do so efficiently enough to be useful at real scale. PLN is the engine meant to do this. The barrier to it doing so well is not the logic itself, which is mature and well specified, but the search process that decides which of PLN's many legal next steps to actually take at each point in a derivation. This project targets that search process directly, inside Hyperon's own reasoning core, rather than building an application on top of it.

3. Background

3.1 The AtomSpace and MeTTa

The AtomSpace is Hyperon's typed knowledge hypergraph; MeTTa is the language used to write, type-check, and reduce (evaluate) expressions against it. Everything PLN reasons over is represented as typed atoms in this store.

3.2 What PLN is and why it exists

PLN combines probabilistic and logical reasoning so the AtomSpace can go beyond stored facts to derived facts, propagating uncertainty (truth values — strength and confidence) through each derivation step. Its rules include deduction, induction, abduction, and various term-logic combination rules, each with a formula for combining premise truth values into a conclusion truth value. The [PLN Repo](#) in MeTTa already supports deduction, induction, abduction, negation, and revision, with tasks and beliefs managed in queues for memory control and prioritization, and with performance strong enough to compile toward real-time use.

3.3 Why the hard part is search, not logic

The rules themselves are well specified. The problem is that, given even a modest AtomSpace, the number of (rule, premise-set) pairs eligible for application at each derivation step is large, and most of them lead nowhere useful. A syntactically simple inference can be nearly worthless if its resulting confidence is low, while a more complex chain can be exactly the missing link toward a valuable conclusion — so simple static

or exhaustive search strategies perform poorly specifically because of PLN's probabilistic, truth-value-propagating nature, not despite it.

3.4 What the industry has tried

- Static/heuristic rule ordering — hand-tuned priority lists; brittle, does not generalize, requires constant manual retuning as the AtomSpace grows.
- Exhaustive or breadth-limited search — correct but does not scale; combinatorics dominate quickly.
- Learned premise/tactic selection in classical theorem provers (e.g. graph-neural or transformer-based rankers guiding Lean/Isabelle/Coq-style proof search) — validates the general pattern of a learned model ranking proof-search moves, but built for crisp, non-probabilistic logics, with no connection to an AtomSpace/MeTTa representation.
- ECAN — spreads “importance” across atoms generally (Short/Long-Term Importance, Hebbian learning), which is valuable but is not a proof-search-specific mechanism; it does not rank inference steps, only atom salience.

3.5 Existing prior art and current feasibility

Geisweiller's own framing ties programmable inference control to minimal-MeTTa — described in Hyperon's own documentation as an attempt to create an assembly language for MeTTa, minimal enough that the full MeTTa interpreter can itself be written in it. minimal-MeTTa is documented, implemented, and actively maintained in the hyperon-experimental repository, and has recently become the subject of an independent formal-verification project (a machine-checked Lean 4 metatheory of the minimal MeTTa interpreter, which reports the minimal MeTTa interpreter and standard library passing Hyperon's own vendored oracle test corpus).

A code-level review of the live PLN implementation found something more directly useful than that dependency: PLN's derivation process (PLN.Derive) is already implemented as a recursive, user-space MeTTa function in `lib_pln.metta` (in the `trueagi-io/PLN` repository, not `hyperon-experimental` — PLN was split out into its own repository in 2025), and task selection and queue pruning already run through two small, isolated functions, `PriorityRank` and `PriorityRankNeg`, both hard-coded to return raw confidence and nothing else. This has been confirmed directly against the live repository, including exact line numbers and a working precedent for the external-call mechanism needed to replace them (§6.1) — substantially de-risking the project relative to earlier drafts, which treated this as gated on minimal-MeTTa's lower-level reduction hooks maturing further.

4. Problem Statement & Gaps

PLN's inference search has no strong prioritization mechanism. This limits both the depth of reasoning practically achievable (search stalls before reaching non-obvious conclusions) and its efficiency (compute wasted on low-value paths) — bottlenecking every downstream Hyperon capability that depends on derived, not merely stored, knowledge.

Gap (named by Hyperon's own PLN lead / community sources)	What PRISM adds
No programmable inference control — explicitly named as the next required capability once minimal-MeTTa is completed	A learned ranking layer over PLN's own legal-candidate generation, hooked in exactly at the reduction-control point minimal-MeTTa is meant to expose
Static/exhaustive search does not scale with AtomSpace size	Best-first / beam search using learned usefulness scores, spending compute on promising steps only

Gap (named by Hyperon's own PLN lead / community sources)	What PRISM adds
No existing mechanism distinguishes structurally simple but low-confidence steps from valuable ones	Ranker is trained/prompted specifically on structure and truth-value/uncertainty signals together, not structure alone
Learned proof-search guidance exists for crisp logics (Lean/Isabelle-style) but not for PLN's probabilistic setting	First adaptation of the learned-ranking pattern to PLN's uncertain, truth-value-propagating rule set

4.1 Prioritization Rationale (Stage 3)

Criterion	Assessment
Criticality	Very high — this is a bottleneck in Hyperon's core reasoning engine, which every other Track A capability (planning, concept formation, knowledge integration) ultimately depends on. It is also, distinctly, a capability the PLN team itself has publicly named as the next thing needed.
Industry Gap	Large and directly evidenced: learned search guidance exists for classical theorem provers, but nothing comparable exists for PLN's probabilistic setting, and nothing integrates with MeTTa/AtomSpace.
Novelty	High — this is the first working instantiation, to our knowledge, of learned inference-step ranking for a probabilistic term logic integrated with an AtomSpace. It is also, unusually for a competition entry, targeting a capability the relevant lead researcher has explicitly said does not yet exist.
Feasibility	High and now substantially confirmed, not just plausible — the exact hook (PriorityRank/PriorityRankNeg in lib_pln.metta), the exact call mechanism (PeTTa's py.call), and a working production precedent for that mechanism are all verified directly, not inferred. The remaining open items (py.call's latency at Tier 1's call frequency, and its failure-mode behavior) are narrow, testable engineering questions, not open feasibility risks (see §9).
Impact	High if successful — a faster, smarter PLN benefits every future Hyperon application at iCog, not only this project's demo domain.
Strategic Relevance	Very high — a working contribution to PLN itself is a contribution to Hyperon's core, and directly answers a capability gap named by the engine's own lead developer.

4.2 AGI Relevance

An AGI-relevant reasoning system must scale its inference to real-world-sized knowledge bases without either exhaustive search or brittle, hand-tuned heuristics. Learning to prioritize which reasoning steps are worth taking — rather than only executing a fixed reasoning procedure — is a component of general intelligence in its own right, arguably more central than any single downstream application (planning, dialogue, tool use) built on top of a reasoning engine. This project targets that capability directly, inside Hyperon's actual reasoning core, rather than building something adjacent to it. Because PLN's derivations underlie any future Hyperon capability that requires going beyond stored facts, an improvement here compounds across every future Track A project, not only this one.

5. System Overview — What PRISM Actually Is

Explained from first principles, before any interface detail: imagine PLN in the middle of building a derivation — a chain of reasoning steps from known facts toward some conclusion. At any point in that chain, PLN can typically see several different legal next moves it's allowed to make — different rules, applied to different combinations of facts it already has. Today, PLN either tries these in a fixed order, or tries all of them, because nothing tells it which ones are actually likely to be worth pursuing. PRISM's entire job is to look at

that list of legal next moves and say, before any of them are actually taken, “these two are probably worth trying first; these others probably aren't.”

The analogy: PLN is a careful, rule-following calculator — it will only ever take a step that's logically legal, and it computes the result of that step exactly and correctly. PRISM is an experienced advisor standing next to it, who has seen enough derivations to have a sense for which of the many legal next steps tend to actually lead somewhere, and says so — but never touches the calculator's arithmetic. The calculator's answer is never influenced by the advisor's opinion; only which calculation gets tried first is.

What PRISM is not: it never proposes a conclusion, never computes a truth value, and never gets to skip PLN's own validity checking. If PRISM ranked a legal step highly and that step turns out to be true or useful, that truth value was still computed entirely by PLN's own formulas, from the premises PLN itself selected — PRISM only affected the order in which legal steps were attempted.

5.1 How PRISM works, step by step

- Frontier generation (PLN's job, unchanged): for the current point in a derivation, PLN enumerates the (rule, premise-set) pairs that are legally applicable right now — this is exactly what PLN already does internally to decide what it's allowed to do next.
- Candidate description (new): each legal candidate is written out in a simple, readable form — which rule, which premises, what their current truth values are, and (if there's a specific goal) what conclusion is being sought.
- Ranking (new, this is PRISM's core function): candidates are scored by how likely they are to be useful — either “likely to lead toward the specific goal” or, for open-ended reasoning, “likely to produce an interesting, high-confidence new fact.” This scoring is deliberately split into a fast pass and a slow pass rather than one uniform check — §6.2 explains why and how.
- Constrained execution (PLN's job, unchanged in mechanism, changed in which candidates reach it): only the top-ranked candidates are actually applied. PLN computes the resulting truth value exactly as it always would — nothing about how a step's result is computed is different.
- Update and repeat: the newly derived fact is added to the AtomSpace, the list of legal next moves is regenerated, and the process continues until the goal is reached, the compute budget runs out, or there are no more legal moves left to try.

Stated plainly: PRISM changes which legal move PLN tries first. It never changes what “legal” means, and it never changes the arithmetic of what a move actually produces.

6. Technical Approach

6.1 Grounding the hook in the real implementation

Earlier drafts of this proposal treated exposing PLN's candidate frontier as an open risk gated on lower-level minimal-MeTTa infrastructure. A code-level review of the live PLN implementation found this is not necessary: derivation is already implemented as a recursive, user-space MeTTa function (PLN.Derive) in `lib_pln.metta`, and task selection and queue pruning already run through two small, isolated, drop-in-replaceable functions — `PriorityRank` and `PriorityRankNeg` — both currently hard-coded to return raw confidence and nothing else, with no structural signal, learned model, or external call of any kind. This has been confirmed directly against the live repository, including the exact line numbers, so it is treated here as verified rather than as an open risk (see §9 for what remains genuinely open).

The external-call mechanism this project needs — letting `PriorityRank` invoke a trained scorer instead of returning raw confidence — is confirmed working via PeTTa's `py.call` grounded-function mechanism, which has been used directly, in production previously.

The goal-threading gap (a real design decision, not just documentation): `PLN.Derive`'s actual signature — `($Tasks $Beliefs $steps $maxsteps $taskqueuesize $beliefqueuesize)` — takes no goal parameter, and

PLN.Query, which does receive the query term, never forwards it down into PLN.Derive. As written, PriorityRank has nothing to compute goal-relevance against, which earlier drafts of this document did not account for. This project threads an OPTIONAL \$Goal parameter through PLN.Query → PLN.Derive → the ranking functions, defaulting to “no goal” rather than being required — both because many existing call sites don't supply one and shouldn't break, and because a scorer call can itself fail (a missing goal, a malformed sentence, a py.call error) and needs to degrade safely to undirected, confidence-based ranking rather than halt derivation (see §9).

The exact substitution, correcting the base-case collision and py.call's real calling convention:

```
;; Current, naive hook (lib_pln.metta:362-363)
(= (PriorityRank (Sentence ($x (stv $f $c)) $Ev1)) $c)
(= (PriorityRank ()) -99999.0)

;; PRISM's replacement — optional goal, explicit empty-candidate guard preserved,
;; falls back to raw confidence if no goal / scorer failure
(= (PriorityRank (Sentence ($x (stv $f $c)) $Ev1) $Goal)
    (if (or (== $Goal ()) (py.call.failed))
        $c
        (py.call scorer score_candidate ((Sentence ($x (stv $f $c)) $Ev1) $Goal))))
(= (PriorityRank () $Goal) -99999.0)

;; illustrative only — the real py.call calling convention (module/function/args
;; shape, and how a Python-side failure is actually surfaced) is confirmed against
;; PeTTa py.call usage before this is implemented, not assumed here
```

Two further corrections to the original hook design, both confirmed directly against lib_pln.metta's actual code: PriorityRankNeg needs the identical optional-goal and fallback treatment, since it is called just as often (via LimitSize, see §6.2's caching note below); and because LimitSize calls BestCandidate repeatedly — once per evicted item — to trim a queue, an uncached scorer would re-cross the Python FFI boundary many times per single derivation step. Scores are therefore memoized on the Sentence atom at first computation (or batched at queue-insertion time) rather than recomputed on every LimitSize comparison — this is treated as part of the core implementation, not an optional optimization, given how directly it compounds with the latency risk already flagged in §9.

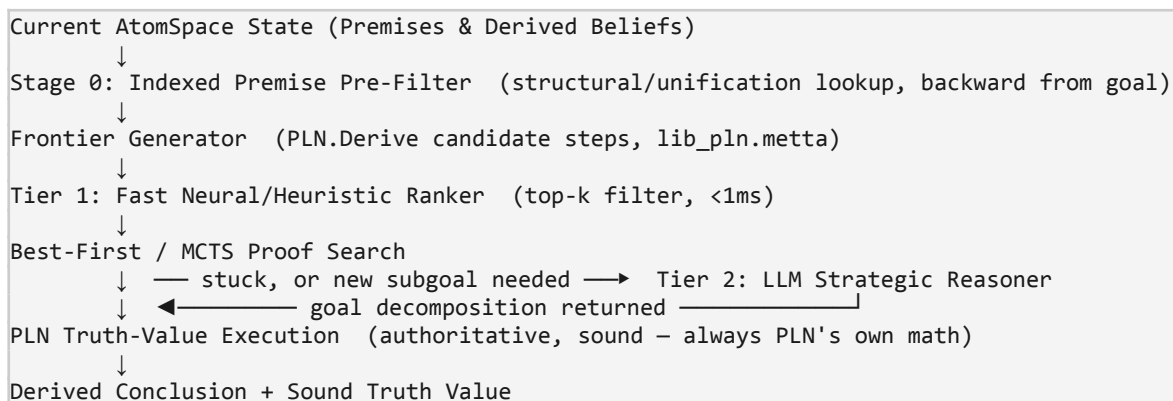
6.2 A staged guidance architecture: indexed pre-filtering + two-tier scoring (not a per-candidate LLM call)

The first design of this proposal called a language model to rank every candidate in the search frontier. This does not scale: MeTTa evaluates inference rules in microseconds, while an LLM call takes on the order of hundreds of milliseconds — if a derivation step has on the order of a hundred or more legal candidates, ranking each with a full LLM call would make proof search take minutes for problems that should take a small fraction of a second. A further code-grounded review raised a second, earlier bottleneck: binary rules (Deduction, Induction, Abduction) generate a combinatorial Cartesian product of eligible premise pairs, so in a realistic AtomSpace the raw candidate frontier can reach hundreds or thousands of pairs before any scoring happens at all — meaning frontier generation itself, not just ranking, has to be kept cheap. PRISM is therefore structured in three stages, not two, matching the shape used by AlphaZero-style search and by modern automated theorem provers such as LeanCopilot, which similarly avoid calling a large model on every micro-step and avoid materializing an unbounded candidate set in the first place:

- Stage 0 — indexed premise pre-filter (before any scoring): rather than letting the frontier generator materialize the full Cartesian product of eligible premise pairs for binary rules, premises are indexed by structural/unification pattern and looked up backward from the current goal or subgoal, so the candidate set that ever reaches Tier 1 is already narrowed. Note: a direct review of PLN-main's existing examples and tests found none exceed the default queue sizes (task queue 10, belief queue 100), and most clamp well below that — so this specific bottleneck has not been demonstrated to actually occur at any scale currently tested in the codebase. Given that, and given that Stage 0 requires directly modifying PLN.Derive's internal derivation step — replacing the unconditional (superpose \$Beliefs) at lib_pln.metta:382–388 with an indexed/goal-aware lookup (unlike the

PriorityRank/PriorityRankNeg substitution in §6.1, which touches nothing else in PLN.Derive) — Stage 0 is treated as conditional, not committed upfront: it is built only if the frontier-size measurement in §7.3 shows it is actually needed once the benchmark domains (§7.1) are run against the unmodified system first.

- Tier 1 — fast policy/heuristic scorer (sub-millisecond): v1 is a symbolic/structural heuristic (term-unification overlap with the goal, discounted by confidence — no training data required to start); v2, trained once enough search traces exist (§8, week 11), is a small dense embedding model or graph neural network over atom representations. Whichever version, this tier runs on every single derivation step and is what actually keeps search fast; its output is a numeric score, never an LLM estimating relevance or confidence from text (see §6.5).
- Tier 2 — strategic LLM reasoner (invoked sparingly): an LLM is called only for higher-level goal decomposition or tactical guidance — specifically when the fast search stalls (no shortlisted candidate leads anywhere after several steps) or when a new subgoal needs to be formed (see §6.5). It is never called per-candidate, and it never computes or estimates a PLN truth-value — that stays entirely inside PLN's own formulas regardless of which tier proposed the step.



This is what makes the calculator/advisor analogy in §5 hold up under real load: Stage 0 keeps the advisor from ever being handed an unmanageable pile of options, Tier 1 is fast enough to consult on every remaining step, and only Tier 2 calls in a second, slower opinion when genuinely needed.

6.3 A formal search algorithm, not “best-first” left vague

Rather than an unspecified best-first search, the search is defined as a learned A*-style best-first search with evaluation function $f(n) = g(n) + h(n)$, where $g(n)$ is the accumulated rule cost and truth-value confidence degradation along the path so far, and $h(n)$ is Tier 1's learned heuristic estimate of distance from the current derivation state to the goal G . Where a policy/value framing is more natural for a subtree, the same search is equivalently expressible in AlphaZero-style terms: rule selection as a policy $\pi(a|s)$ and proof-success likelihood as a value $V(s)$, with Tier 1 providing both.

6.4 Bidirectional search: forward chaining is not enough alone

Forward chaining alone — even with a good ranker — struggles when the goal is distant from current knowledge, since the chain has to be discovered from the premise side with no information about the target. PRISM's search therefore supports backward chaining (decomposing the goal into subgoals nearer to known facts) alongside forward chaining, with Tier 2's LLM specifically responsible for subgoal decomposition when the forward search stalls, and a bidirectional “meet in the middle” termination condition where a forward-derived fact and a backward-derived subgoal connect.

6.5 Candidate representation

Both tiers work over the same structured, readable candidate description — not raw AtomSpace serialization — but they consume it differently: Tier 1 converts these features into vectors and produces a numeric score

via a trained model's forward pass (never a text-reasoning estimate), while Tier 2, when invoked, reads the description as text for strategic judgment:

```
;; Illustrative candidate description
Rule: deduction
Premises: (Inheritance Alice Ancestor-of-Bob) [0.9, 0.8]
          (Inheritance Bob Ancestor-of-Carol) [0.85, 0.9]
Goal: (Inheritance Alice Ancestor-of-Carol)
→ Tier 1: numeric score from the trained scorer's forward pass over these features
  (a dot-product/GNN computation, not an LLM estimating relevance or confidence from text)
→ Tier 2 (only if stalled): does this goal need decomposing into subgoals?
```

6.6 Two ranking modes

- Goal-directed: how likely a step is to be on a productive path toward the target conclusion — closest to premise selection in classical automated theorem proving.
- Undirected / exploratory: how likely a step is to yield a novel, high-confidence, non-trivial derived fact — useful for building out an AtomSpace's derived knowledge generally, closer to open-ended concept discovery.

6.7 Engineering stack

- Stage 0: an index over the AtomSpace keyed by structural/unification pattern (predicate and type signature), supporting backward lookup from a goal or subgoal, built and maintained alongside the existing PLN.Derive path in lib_pln.metta (§6.1).
- Python orchestration around PLN.Derive, called from MeTTa via PeTTa's py.call grounded-function mechanism (§6.1) — the same mechanism already used in production for other currently maintained components.
- Tier 1: v1 is a symbolic/structural scoring rule (no training data needed); v2 (week 11) is a small embedding or graph neural network model trained on candidate-outcome traces logged during weeks 5–10 (§7, §8) — both exposed as a Python scorer invoked via py.call from a replaced PriorityRank, with results memoized to avoid redundant FFI calls during LimitSize's repeated comparisons (§6.1).
- Tier 2: an open-weight LLM, invoked only for subgoal decomposition and stall recovery via the same py.call mechanism — no fine-tuning required for v1 (fine-tuning on the system's own successful search traces remains an explicit stretch goal, §8).
- Derivation-tree bookkeeping (candidate frontier, best-first priority queue, forward/backward search state) in Python for iteration speed.

7. Evaluation Plan

7.1 Domains

The existing toy examples shipped with PLN (PLN/examples/ — Smokes, FlyingRaven, Toothbrush) are used only for initial correctness checks during development, not as the reported benchmark — they are too small to demonstrate a real search-efficiency improvement. The actual evaluation domains are:

- Synthetic transitive-inference chains at depth $D \in [5, 20]$, where the correct proof path and its length are known exactly, letting search efficiency be measured precisely as a function of depth.
- Knowledge-graph multi-hop reasoning, using a subset of a standard multi-hop benchmark (e.g. FB15k-237) or a biomedical ontology subset translated into MeTTa atoms — chosen for realistic scale and branching factor, not hand-crafted convenience.
- A scaling test: the same domains at increasing AtomSpace size, to check whether wasted search grows sub-linearly relative to unguided search as domains get larger.

- A cross-domain generalization test: rank on one domain family, evaluate on a structurally different one, to check whether the ranker is learning general “useful inference step” judgment rather than overfitting.

7.2 Baselines

Baseline	What it is	Used to isolate
Unguided PLN	Breadth/depth-limited search, no ranking	Value of ranking at all
Static-heuristic PLN	Hand-ordered rule priority (if available in the current codebase)	Value of a learned ranker over a hand-tuned one

7.3 Core metrics

- Frontier size before vs. after Stage 0 indexing, on the KG multi-hop domain specifically — the direct check that pre-filtering actually prevents the Cartesian-product blow-up identified in §6.2, independent of how fast Tier 1 scores whatever it receives.
- Wall-clock speedup to reach a valid proof, guided vs. unguided — measured end-to-end, including Stage 0/Tier 1/Tier 2 overhead together, not just rule-application count, so the full pipeline's latency claim (§6.2) is checked as a whole, not stage by stage in isolation.
- Proof-tree node expansion reduction — fraction of applied rules that do not end up contributing to the final accepted conclusion(s), guided vs. unguided — target 40–60% reduction.
- Success rate under a fixed step budget, at each tested depth D — the most direct evidence of whether guidance helps specifically as problems get harder.
- Cross-domain generalization: meaningfully above random/static-heuristic baseline, even if below in-domain performance.
- Soundness check (should hold at 100% by construction): every conclusion PLN outputs is independently re-verifiable by re-running PLN's own rule formulas on the recorded premises — confirming the ranker never influenced validity, only order.
- Undirected-mode “interesting conclusion” recall against a hand-curated set of non-obvious derivable facts — target $\geq 50\%$ within a fixed compute budget.

8. Timeline (12 Weeks)

Weeks	Focus
1	Benchmark py.call's actual latency at Tier-1-realistic call frequency (§9); implement and test the PriorityRank/PriorityRankNeg swap directly against lib_pln.metta (§6.1), including a fallback path for scorer-side errors
2	Define synthetic evaluation domains (transitive chains $D \in [5,20]$, KG multi-hop subset) and hand-curate “interesting conclusion” gold sets; establish unguided and static-heuristic baselines
3–4	Implement Tier 1 v1 as a fast symbolic/structural heuristic (e.g. term-unification overlap with the goal, discounted by confidence) — not a trained model yet, since no search traces exist to train one on. Implement Stage 0 only if week 1–2's frontier-size measurement showed it was needed. Wire in the optional \$Goal threading (§6.1).
5–6	Wire Tier 1 v1 into the formal A*/best-first search loop (§6.3); goal-directed ranking mode first; begin logging search traces for later Tier 1 training
7	Controlled-domain evaluation of forward-only guidance with the symbolic Tier 1 heuristic; iterate on its scoring rules

Weeks	Focus
8	Implement Tier 2 (LLM strategic reasoner) for stall recovery and subgoal decomposition; implement backward chaining and the bidirectional termination condition (§6.4); continue logging search traces
9	Scaling test and cross-domain generalization test using Tier 1 v1 (symbolic) + Tier 2; end-to-end wall-clock benchmarking of the full pipeline
10	Soundness verification pass; robustness checks on edge-case candidate sets, including scorer-failure fallback paths (§6.1, §9)
11	Train Tier 1 v2 (embedding/GNN model) on the search traces logged in weeks 5–10, replacing the symbolic heuristic; compare v1 vs. v2 directly rather than assuming the learned model is strictly better
12	Write-up, benchmark report, demo against unguided/static baselines

9. Risks & Mitigations

Risk	Mitigation
Per-candidate LLM calls make search slower than no guidance at all — the flaw identified in the original draft's technical approach	Resolved by design in §6.2: an LLM is never called per-candidate. Tier 1 (sub-millisecond target) handles every step; Tier 2 (LLM) is invoked only on stall or subgoal formation, and wall-clock speedup (§7.3) is measured end-to-end specifically to verify this holds in practice, not just in design
py.call's latency at Tier 1's call frequency (every single task selection) is unverified — prior production use reported no noticeable slowdown, but that was not rigorously benchmarked and may not reflect Tier 1's much higher call frequency	Benchmark py.call's actual latency at realistic call frequency as the very first task in week 1 (§8), before any scorer logic is built — if it is too slow to call on every task selection, the fallback is calling it only every k selections or restricting it to Tier 2's sparser strategic calls instead of Tier 1's fast path
py.call has no confirmed graceful failure mode — a Python-side error (e.g. a missing virtual environment or import failure) surfaces directly as a raw error rather than falling back cleanly, which could halt PLN.Derive entirely if the scorer script fails mid-derivation	Wrap the scorer call in an explicit try/fallback at the MeTTa or Python layer so a scorer failure degrades to the original raw-confidence PriorityRank rather than halting derivation; this fallback path is tested explicitly as part of the soundness/robustness pass (§7.3, week 10)
Frontier generation itself (the Cartesian product of eligible premise pairs for binary rules like Deduction/Induction/Abduction) may bottleneck at the larger scale this project's benchmarks target, even though no existing PLN-main example currently reaches that scale (§6.2)	Stage 0 (§6.2) is built defensively for this; frontier size before/after indexing (§7.3) is measured directly on the realistic KG multi-hop domain to confirm whether it is actually load-bearing at the scale being tested
The lib_pln.metta / PriorityRank hook (§6.1) does not behave exactly as expected once modified directly	Already substantially de-risked: the exact function, line numbers, and a working substitution mechanism (py.call, with prior production precedent) are confirmed. Week 1 of §8 is a direct implementation and test of the swap, not an open-ended investigation
Tier 1 v1 (symbolic heuristic) is too weak to produce a useful shortlist,	Validate v1's shortlist quality directly (does the true best move survive the top-k cut) against the controlled domains in week 7 before Tier 2

Risk	Mitigation
pushing more load onto the slower Tier 2 than intended	is built in week 8; v2's trained model (week 11) is compared directly against v1 rather than assumed to be strictly better (§8)
Backward chaining / bidirectional search (§6.4) adds more implementation complexity than forward-only guidance	Forward-chaining-only guidance is treated as the v1 floor and is independently demonstrable; backward chaining and the bidirectional meet-in-the-middle condition are added once forward guidance is validated, not built simultaneously
Ranker has no PLN-specific training signal initially	Start with few-shot prompting for Tier 2 and a small labeled candidate-outcome set for Tier 1 (§6.7); treat fine-tuning on search traces as an explicit stretch goal (week 11), not a dependency for core success
Soundness regression (guidance somehow influencing conclusions, not just order)	Structural guarantee by design (neither tier's output reaches truth-value computation), plus explicit soundness-check testing (§7.3) as a release gate

10. Expected Contribution to iCog

- A reusable, learned inference-control layer for PLN — directly improving the core reasoning engine any future Hyperon application depends on.
- A documented, benchmarked interface into PLN's proof-search internals, useful groundwork for any future work touching PLN.
- A concrete answer to a capability gap named directly by PLN's own lead developer, not an assumed or inferred one.
- Empirical data (waste ratios, scaling curves, generalization results) informing whether and how far to invest further in learned PLN search control at iCog.

11. References

Geisweiller, N. Commentary on PLN in MeTTa and the requirement for programmable inference control tied to minimal-MeTTa completion, as referenced in “OpenCog Hyperon: A Framework for AGI at the Human Level and Beyond,” arXiv:2310.18318.

trueagi-io/hyperon-experimental (GitHub). docs/minimal-metta.md — the working specification and implementation of minimal MeTTa referenced throughout §3.5 and §6.1.

MesTTo/LeaTTo (GitHub). A machine-checked Lean 4 formalization of Hyperon's minimal MeTTa interpreter, cited as evidence of minimal-MeTTa's current maturity (§3.5).

SingularityNET Ambassadors, “Hyperon: Building the Future of Neural Symbolic Evolutionary AGI” (Medium). Background on PLN 0.5's supported inference types and the PRIMUS cognitive architecture, referenced in §3.2.

“Toward a Formal Model of Cognitive Synergy” (arXiv:1703.04361). Background on ECAN, pattern mining, and concept formation as related but distinct cognitive processes, referenced in §3.4.

DEEP Funding, “Framework for evaluating approaches to attention allocation.” Evidence that adjacent search/attention problems in Hyperon are currently recognized as open and fundable, referenced for context in §3.4.

trueagi-io/PLN, lib_pln.metta. The live, production PLN implementation — specifically the PLN.Derive function and the PriorityRank/PriorityRankNeg scoring functions — referenced throughout §6 as the concrete integration point for this project. Note: PLN was split out into its own repository (trueagi-io/PLN) in

2025 and is distinct from trueagi-io/hyperon-experimental and the older, superseded trueagi-io/pln-experimental.

DeepMind, AlphaProof / AlphaZero-style policy–value search. Referenced in §6.2–§6.3 as the precedent for the two-tier fast-filter/strategic-reasoner architecture and the policy/value framing of the search algorithm.

LeanCopilot (and related learned-tactic systems for Lean/Isabelle-style provers). Referenced in §6.2 as precedent for reserving large-model calls for strategic guidance rather than per-step scoring.

12. Appendix — Illustrative Scenario

AtomSpace contains family-relation facts and a partial derivation goal: is Alice an ancestor of Carol?

- PLN's frontier generator enumerates legal next steps: several transitivity applications over parent-of links, an irrelevant sibling-relation rule, and a low-confidence inheritance-rule application.
- PRISM, given the readable candidate descriptions and the goal, ranks the transitivity-over-parent-of steps highest and the sibling-relation step lowest — correctly judging it irrelevant to an ancestry goal.
- PLN applies only the top-ranked candidates, reaching the ancestor conclusion in far fewer total rule applications than unguided breadth-first search would have required.

The eventual truth value on “Alice is an ancestor of Carol” is computed entirely by PLN's own formulas, unaffected by PRISM's involvement in getting there faster.